CIFP AVILÉS · CENTRO INTEGRADO DE FORMACIÓN PROFESIONAL
Departamento de Informática y Comunicaciones · Principado de Asturias

FP

Solucionario Técnico: Bloque 2

Ficheros Binarios, Acceso Posicional Directo (RAF) y Serialización Segura

MÓDULO PROFESIONAL	CICLO Y UNIDAD	MODALIDAD / ENTORNO	DESTINATARIO / CARÁCTER
Acceso a Datos (0486)	**DAM 2º Curso · UT1**	**Presencial / Distancia · Java 21**	**Docente · Solución Oficial Comentada**

ALINEACIÓN CURRICULAR (RA1: MANEJO DE FICHEROS)

RA1: Manejo de Ficheros | CE1.b (Formas de acceso) | CE1.c (Recuperar información) | CE1.d (Almacenar información) | CE1.f (Excepciones)

Módulo: Acceso a Datos | **Ciclo**: DAM2 | **Unidad**: UT1
Hoja de Enunciados: Ver Enunciados para Alumnos
Docente: Sergio Capdevila Díez · CIFP Avilés

Solución Ejercicio 1: Caja Negra de Flota de Vehículos (`CajaNegraFlota.java`)

```
package soluciones_b2;

import java.io.*;
import java.nio.charset.StandardCharsets;

public class CajaNegraFlota {
    private static final String RUTA_FICHERO = "datos/flota_gps.dat";

    public static void registrarPosicion(int idVehiculo, String matricula, double lat,
double lon, double vel, boolean enRuta) {
        File f = new File(RUTA_FICHERO);
        f.getParentFile().mkdirs();

        try (DataOutputStream dos = new DataOutputStream(
                new BufferedOutputStream(new FileOutputStream(f, true)))) {

            dos.writeInt(idVehiculo);
            dos.writeUTF(matricula);
            dos.writeDouble(lat);
            dos.writeDouble(lon);
            dos.writeDouble(vel);
            dos.writeBoolean(enRuta);

            System.out.printf("✅ Telemetría registrada: Vehículo %d (%s) - %.1f
```

```
km/h%n", idVehiculo, matricula, vel);

        } catch (IOException e) {
            System.err.println("Error al escribir telemetría binaria: " +
e.getMessage());
        }
    }

    public static void generarInformeFlota() {
        File f = new File(RUTA_FICHERO);
        if (!f.exists()) {
            System.out.println("No hay datos de telemetría registrados.");
            return;
        }

        int totalRegistros = 0;
        int excesosVelocidad = 0;


System.out.println("=============================================================================="
);
        System.out.println("            AUDITORÍA DE FLOTA: EXCESOS DE VELOCIDAD (>120
km/h) ");

System.out.println("=============================================================================="
);

        try (DataInputStream dis = new DataInputStream(
                new BufferedInputStream(new FileInputStream(f)))) {

            while (true) {
                int id = dis.readInt();
                String mat = dis.readUTF();
                double lat = dis.readDouble();
                double lon = dis.readDouble();
                double vel = dis.readDouble();
                boolean enRuta = dis.readBoolean();
                totalRegistros++;

                if (vel > 120.0) {
                    excesosVelocidad++;
                    System.out.printf("⚠️ ALERTA: Vehículo %d [%s] a %.2f km/h en pos
(%.4f, %.4f) - En ruta: %b%n",
                            id, mat, vel, lat, lon, enRuta);
                }
            }

        } catch (EOFException e) {
            System.out.println("-------------------------------------------------------
-----------");
            System.out.printf("Fin de informe: %d tramas procesadas (%d infracciones
detectadas).%n",
                    totalRegistros, excesosVelocidad);
        } catch (IOException e) {
            System.err.println("Error leyendo fichero binario: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        new File(RUTA_FICHERO).delete();   // empezamos de cero en cada prueba
        registrarPosicion(1, "1234-BBB", 43.556, -5.924, 115.0, true);
```

```
        registrarPosicion(2, "5678-CCC", 43.535, -5.661, 132.5, true); // Exceso
        registrarPosicion(3, "9012-DDD", 43.361, -5.849, 90.0, false);
        registrarPosicion(2, "5678-CCC", 43.540, -5.670, 128.0, true); // Exceso

        generarInformeFlota();

        // Comparación con un CSV equivalente (mismos datos, una línea por trama)
        String csv = "1;1234-BBB;43.556;-5.924;115.0;true\n"
                   + "2;5678-CCC;43.535;-5.661;132.5;true\n"
                   + "3;9012-DDD;43.361;-5.849;90.0;false\n"
                   + "2;5678-CCC;43.54;-5.67;128.0;true\n";
        System.out.printf("Tamaño binario: %d bytes · CSV equivalente: %d bytes%n",
                new File(RUTA_FICHERO).length(),
csv.getBytes(StandardCharsets.UTF_8).length);
    }
}
```

▶️ Salida real (Java 21, configuración regional española)

```
✅ Telemetría registrada: Vehículo 1 (1234-BBB) - 115,0 km/h
✅ Telemetría registrada: Vehículo 2 (5678-CCC) - 132,5 km/h
✅ Telemetría registrada: Vehículo 3 (9012-DDD) - 90,0 km/h
✅ Telemetría registrada: Vehículo 2 (5678-CCC) - 128,0 km/h
======================================================================
             AUDITORÍA DE FLOTA: EXCESOS DE VELOCIDAD (>120 km/h)
======================================================================
⚠️ ALERTA: Vehículo 2 [5678-CCC] a 132,50 km/h en pos (43,5350, -5,6610) - En ruta: true
⚠️ ALERTA: Vehículo 2 [5678-CCC] a 128,00 km/h en pos (43,5400, -5,6700) - En ruta: true
----------------------------------------------------------------------
Fin de informe: 4 tramas procesadas (2 infracciones detectadas).
Tamaño binario: 156 bytes · CSV equivalente: 142 bytes
```

> **Nota para el aula:** cada trama ocupa 39 bytes en binario: 4 (`int`) + 2 + 8 (`writeUTF` de una matrícula de 8 caracteres) + 3 × 8 (`double`) + 1 (`boolean`) y su línea CSV entre 33 y 37 (142 bytes en total frente a 156): con coordenadas de 3 decimales **el binario no ocupa menos**. Compensa cuando los números son grandes o tienen muchos decimales (`43.55612345678` ocupa 14 caracteres en texto y 8 bytes en binario), además de leerse sin `parseDouble` . Es justo lo que pide comentar el enunciado.

Solución Ejercicio 2: Gestor de Almacén Directo en Disco (`AlmacenRAF.java`)

```
package soluciones_b2;

import java.io.File;
import java.io.IOException;
import java.io.RandomAccessFile;

public class AlmacenRAF {
    // 4B (id) + 16B (8 chars codigo) + 20B (10 chars nombre) + 4B (stock) = 44 Bytes
    public static final int TAM_REGISTRO = 44;
```

```
    private final File archivo;

    public AlmacenRAF(String ruta) {
        this.archivo = new File(ruta);
        this.archivo.getParentFile().mkdirs();
    }

    private String ajustarTexto(String texto, int longitudFija) {
        if (texto == null) texto = "";
        if (texto.length() > longitudFija) {
            return texto.substring(0, longitudFija);
        }
        StringBuilder sb = new StringBuilder(texto);
        while (sb.length() < longitudFija) {
            sb.append(' ');
        }
        return sb.toString();
    }

    public void insertarArticulo(int posicion, int id, String codigo, String nombre, int
stock) throws IOException {
        try (RandomAccessFile raf = new RandomAccessFile(archivo, "rw")) {
            long offset = (long) posicion * TAM_REGISTRO;
            raf.seek(offset);

            raf.writeInt(id);
            raf.writeChars(ajustarTexto(codigo, 8));   // 8 * 2 = 16 bytes
            raf.writeChars(ajustarTexto(nombre, 10));  // 10 * 2 = 20 bytes
            raf.writeInt(stock);                      // 4 bytes

            System.out.printf("✅ Artículo insertado en posición %d (offset: %d
bytes)%n", posicion, offset);
        }
    }

    public void actualizarStock(int posicion, int nuevoStock) throws IOException {
        try (RandomAccessFile raf = new RandomAccessFile(archivo, "rw")) {
            // Offset exacto del stock = base + 4 (id) + 16 (codigo) + 20 (nombre) =
base + 40
            long offsetStock = ((long) posicion * TAM_REGISTRO) + 40;
            if (offsetStock + 4 > raf.length()) {
                throw new IndexOutOfBoundsException("La posición " + posicion + " no
existe en el fichero.");
            }

            raf.seek(offsetStock);
            raf.writeInt(nuevoStock);

            System.out.printf("🔄 Stock actualizado en disco para posición %d -> Nuevo
stock: %d%n", posicion, nuevoStock);
        }
    }

    public void consultarArticulo(int posicion) throws IOException {
        try (RandomAccessFile raf = new RandomAccessFile(archivo, "r")) {
            long offset = (long) posicion * TAM_REGISTRO;
            if (offset + TAM_REGISTRO > raf.length()) {
                System.out.println("❌ No existe registro en la posición " + posicion);
                return;
            }
```

```
            raf.seek(offset);
            int id = raf.readInt();

            char[] cCod = new char[8];
            for (int i = 0; i < 8; i++) cCod[i] = raf.readChar();
            String codigo = new String(cCod).trim();

            char[] cNom = new char[10];
            for (int i = 0; i < 10; i++) cNom[i] = raf.readChar();
            String nombre = new String(cNom).trim();

            int stock = raf.readInt();

            System.out.printf("📦 [Pos %d] ID: %d | Código: %s | Nombre: %s | Stock: %d
uds.%n",
                    posicion, id, codigo, nombre, stock);
        }
    }

    public long obtenerTotalArticulos() throws IOException {
        try (RandomAccessFile raf = new RandomAccessFile(archivo, "r")) {
            return raf.length() / TAM_REGISTRO;
        }
    }

    public void truncarAlmacen(int maxRegistros) throws IOException {
        try (RandomAccessFile raf = new RandomAccessFile(archivo, "rw")) {
            long nuevaLongitud = (long) maxRegistros * TAM_REGISTRO;
            raf.setLength(nuevaLongitud);
            System.out.printf("✂️ Archivo truncado a %d registros (%d bytes).%n",
maxRegistros, nuevaLongitud);
        }
    }

    public static void main(String[] args) throws IOException {
        new File("datos/articulos.dat").delete();   // empezamos de cero en cada prueba
        AlmacenRAF almacen = new AlmacenRAF("datos/articulos.dat");

        almacen.insertarArticulo(0, 101, "COD-0001", "Monitor 4K", 15);
        almacen.insertarArticulo(1, 102, "COD-0002", "Teclado Mec", 50);
        almacen.insertarArticulo(2, 103, "COD-0003", "Mouse Pad", 100);

        System.out.println("Antes de modificar:");
        almacen.consultarArticulo(1);

        // Modificamos directamente en disco el stock del registro 1
        almacen.actualizarStock(1, 42);

        System.out.println("Después de modificar:");
        almacen.consultarArticulo(1);

        System.out.println("Total de artículos: " + almacen.obtenerTotalArticulos());
        almacen.truncarAlmacen(2);
        System.out.println("Total de artículos: " + almacen.obtenerTotalArticulos());
        almacen.consultarArticulo(2);
    }
}
```

▶️ Salida real (Java 21)

```
✅ Artículo insertado en posición 0 (offset: 0 bytes)
✅ Artículo insertado en posición 1 (offset: 44 bytes)
✅ Artículo insertado en posición 2 (offset: 88 bytes)
Antes de modificar:
📦 [Pos 1] ID: 102 | Código: COD-0002 | Nombre: Teclado Me | Stock: 50 uds.
🔄 Stock actualizado en disco para posición 1 -> Nuevo stock: 42
Después de modificar:
📦 [Pos 1] ID: 102 | Código: COD-0002 | Nombre: Teclado Me | Stock: 42 uds.
Total de artículos: 3
✂️ Archivo truncado a 2 registros (88 bytes).
Total de artículos: 2
❌ No existe registro en la posición 2
```

> **Nota para el aula:** «Teclado Mec» tiene 11 caracteres y se trunca a 10 («Teclado Me»): es el comportamiento esperado de un campo de longitud fija (apartado 4.3).

Solución Ejercicio 3: Auditoría Bancaria con Append Seguro (`AuditoriaBancaria.java`)

```
package soluciones_b2;

import java.io.*;
import java.time.LocalDateTime;

public class AuditoriaBancaria {

    // 1. Subclase para el modo append: no escribe la cabecera y reinicia las
referencias
    public static class MiObjectOutputStream extends ObjectOutputStream {
        public MiObjectOutputStream(OutputStream out) throws IOException {
            super(out);
        }

        @Override
        protected void writeStreamHeader() throws IOException {
            // No escribe la cabecera 0xACED0005 (el fichero ya la tiene) y emite
TC_RESET
            // para que el lector reinicie su tabla de referencias (evita objetos
cruzados).
            reset();
        }
    }

    // 2. Entidad Serializable con transient
    public static class Transaccion implements Serializable {
        private static final long serialVersionUID = 1L;

        private long id;
        private String cuentaOrigen;
        private String cuentaDestino;
```

```
        private double importe;
        private LocalDateTime fecha;
        private transient String pinAutorizacion; // ¡NO se persiste!

        public Transaccion(long id, String origen, String destino, double importe,
String pin) {
            this.id = id;
            this.cuentaOrigen = origen;
            this.cuentaDestino = destino;
            this.importe = importe;
            this.fecha = LocalDateTime.now();
            this.pinAutorizacion = pin;
        }

        @Override
        public String toString() {
            return String.format("[%s] ID: %d | %s -> %s | %.2f € | PIN RAM: %s",
                    fecha.toLocalTime().withNano(0), id, cuentaOrigen, cuentaDestino,
importe, pinAutorizacion);
        }
    }

    // 3. Método de registro con operador ternario docente
    public static void registrarOperacion(File archivo, Transaccion t) {
        archivo.getParentFile().mkdirs();
        boolean yaExisteConDatos = archivo.exists() && archivo.length() > 0;

        try (FileOutputStream fos = new FileOutputStream(archivo, true);
             ObjectOutputStream oos = yaExisteConDatos
                     ? new MiObjectOutputStream(fos)
                     : new ObjectOutputStream(fos)) {

            oos.writeObject(t);
            oos.flush();
            System.out.println("✅ Transacción registrada: ID " + t.id);

        } catch (IOException e) {
            System.err.println("Error al serializar transacción: " + e.getMessage());
        }
    }

    // 4. Lectura secuencial completa hasta EOFException
    public static void listarHistorial(File archivo) {
        if (!archivo.exists()) {
            System.out.println("No existe archivo de auditoría.");
            return;
        }


System.out.println("=========================================================================="
);
        System.out.println("            HISTORIAL DE AUDITORÍA BANCARIA
(DESERIALIZACIÓN)     ");

System.out.println("=========================================================================="
);

        int contador = 0;
        try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream(archivo))) {
            while (true) {
```

```
                Transaccion t = (Transaccion) ois.readObject();
                contador++;
                System.out.println("  " + t);
            }
        } catch (EOFException e) {
            System.out.println("------------------------------------------------------------
-----------");
            System.out.printf("Auditoría finalizada con éxito: %d transacciones
recuperadas.%n", contador);
        } catch (ClassNotFoundException e) {
            System.err.println("Error: Clase Transaccion no encontrada.");
        } catch (StreamCorruptedException e) {
            System.err.println("💥 ERROR GRAVE: Stream corrupto por doble cabecera (¿se
usó MiObjectOutputStream?): " + e.getMessage());
        } catch (IOException e) {
            System.err.println("Error de lectura: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        File f = new File("datos/transacciones.ser");
        f.delete(); // Limpiar de pruebas previas

        // Simulamos 3 llamadas independientes en distintos momentos
        registrarOperacion(f, new Transaccion(1001, "ES10-0001", "ES10-0002", 350.00,
"1234"));
        registrarOperacion(f, new Transaccion(1002, "ES10-0003", "ES10-0001", 1200.50,
"9999"));
        registrarOperacion(f, new Transaccion(1003, "ES10-0002", "ES10-0005", 75.25,
"0000"));

        listarHistorial(f);
    }
}
```

▶️ Salida real (Java 21, configuración regional española; la hora varía en cada ejecución)

```
✅ Transacción registrada: ID 1001
✅ Transacción registrada: ID 1002
✅ Transacción registrada: ID 1003
======================================================================
             HISTORIAL DE AUDITORÍA BANCARIA (DESERIALIZACIÓN)
======================================================================
  [17:13:55] ID: 1001 | ES10-0001 -> ES10-0002 | 350,00 € | PIN RAM: null
  [17:13:55] ID: 1002 | ES10-0003 -> ES10-0001 | 1200,50 € | PIN RAM: null
  [17:13:55] ID: 1003 | ES10-0002 -> ES10-0005 | 75,25 € | PIN RAM: null
----------------------------------------------------------------------
Auditoría finalizada con éxito: 3 transacciones recuperadas.
```

> **Por qué `reset()`:** cada `ObjectOutputStream` numera los objetos que escribe y el lector lleva la numeración acumulada de todo el fichero. Si en una misma apertura se añadieran dos transacciones que comparten una referencia (el mismo `String` de cuenta, por ejemplo), sin `reset()` la segunda se leería con datos de otra y sin ninguna excepción.

Solución Ejercicio 4: Ranking Binario de un Torneo (`PuntuacionesTorneo.java`)

```
package soluciones_b2;

import java.io.*;
import java.util.ArrayList;
import java.util.List;

public class PuntuacionesTorneo {
    private static final File FICHERO = new File("datos/puntuaciones.dat");

    /** Una partida tal como se lee del fichero. */
    static class Puntuacion {
        String jugador;
        int puntos;
        double segundos;

        Puntuacion(String jugador, int puntos, double segundos) {
            this.jugador = jugador;
            this.puntos = puntos;
            this.segundos = segundos;
        }
    }

    /** Escribe una sesión de partidas: append=false crea el fichero, append=true añade
al final. */
    static void guardarSesion(boolean append, String[] jugadores, int[] puntos, double[]
segundos) {
        try (DataOutputStream dos = new DataOutputStream(
                new BufferedOutputStream(new FileOutputStream(FICHERO, append)))) {
            for (int i = 0; i < jugadores.length; i++) {
                dos.writeUTF(jugadores[i]);     // 2 + N bytes
                dos.writeInt(puntos[i]);        // 4 bytes
                dos.writeDouble(segundos[i]);   // 8 bytes
            }
        } catch (IOException e) {
            System.err.println("Error al guardar la sesión: " + e.getMessage());
        }
        // Tamaño consultado DESPUÉS de cerrar (si no, parte sigue en el búfer)
        System.out.printf("Sesión guardada (%s): %d partidas · fichero de %d bytes%n",
                append ? "añadida" : "nueva", jugadores.length, FICHERO.length());
    }

    /** Lee todas las partidas hasta EOFException. */
    static List<Puntuacion> leerTodas() {
        List<Puntuacion> lista = new ArrayList<>();
        try (DataInputStream dis = new DataInputStream(
                new BufferedInputStream(new FileInputStream(FICHERO)))) {
```

```
            while (true) {                                  // mismo orden que al
escribir
                String jugador = dis.readUTF();
                int puntos = dis.readInt();
                double segundos = dis.readDouble();
                lista.add(new Puntuacion(jugador, puntos, segundos));
            }
        } catch (EOFException e) {
            // fin normal del fichero: no es un error
        } catch (IOException e) {
            System.err.println("Error de lectura: " + e.getMessage());
        }
        return lista;
    }

    public static void main(String[] args) {
        guardarSesion(false, new String[]{"Ana", "Bruno", "Carla", "Diego"},
                new int[]{1200, 980, 1500, 1200}, new double[]{95.5, 80.0, 120.25,
88.0});
        guardarSesion(true, new String[]{"Elena", "Bruno", "Fran"},
                new int[]{1350, 1500, 700}, new double[]{101.0, 110.5, 60.0});

        List<Puntuacion> todas = leerTodas();
        System.out.println("Partidas leídas: " + todas.size());

        // Más puntos primero; a igualdad de puntos, gana quien tardó menos
        todas.sort((a, b) -> a.puntos != b.puntos
                ? Integer.compare(b.puntos, a.puntos)
                : Double.compare(a.segundos, b.segundos));

        System.out.println("🏆 TOP 3 DEL TORNEO");
        for (int i = 0; i < 3 && i < todas.size(); i++) {
            Puntuacion p = todas.get(i);
            System.out.printf("  %d. %-6s %5d puntos  %7.2f s%n", i + 1, p.jugador,
p.puntos, p.segundos);
        }
    }
}
```

▶️ Salida real (Java 21, configuración regional española)

```
Sesión guardada (nueva): 4 partidas · fichero de 74 bytes
Sesión guardada (añadida): 3 partidas · fichero de 130 bytes
Partidas leídas: 7
🏆 TOP 3 DEL TORNEO
    1. Bruno   1500 puntos   110,50 s
    2. Carla   1500 puntos   120,25 s
    3. Elena   1350 puntos   101,00 s
```

Notas para el aula: tamaño calculado a mano: cada partida ocupa (2 + N) + 4 + 8 bytes, con N las letras del nombre (todas ASCII). Sesión 1: 17 + 19 + 19 + 19 = 74 bytes; sesión 2: 19 + 19 + 18 = 56; total 130. El *append* funciona porque `DataOutputStream` no escribe ninguna cabecera: el fichero es solo la secuencia de datos. Con `ObjectOutputStream` el mismo `FileOutputStream(f, true)` daría `StreamCorruptedException` (§4.5). Bruno aparece dos veces en el fichero (980 y 1500): el ranking es de partidas, no de jugadores. Carla y Bruno empatan a 1500: el desempate por tiempo pone primero a Bruno (110,50 s).

Solución Ejercicio 5: Inventario Serializado de un Taller (`InventarioTaller.java`)

```
package soluciones_b2;

import java.io.*;
import java.util.ArrayList;
import java.util.List;

public class InventarioTaller {

    public static class Proveedor implements Serializable {
        private static final long serialVersionUID = 1L;
        private final String cif;
        private final String nombre;

        public Proveedor(String cif, String nombre) {
            this.cif = cif;
            this.nombre = nombre;
        }

        public String getNombre() { return nombre; }
    }

    public static class Pieza implements Serializable {
        private static final long serialVersionUID = 1L;
        private final String ref;
        private final String descripcion;
        private final double precio;
        private final int unidades;
        private final Proveedor proveedor;      // referencia a otro objeto: se
serializa con él
        private transient double valorStock;    // dato derivado: no se guarda, se
recalcula

        public Pieza(String ref, String descripcion, double precio, int unidades,
Proveedor proveedor) {
            this.ref = ref;
            this.descripcion = descripcion;
            this.precio = precio;
            this.unidades = unidades;
            this.proveedor = proveedor;
            this.valorStock = precio * unidades;
        }

        /** Tras deserializar, valorStock vale 0.0 (valor por defecto de un double
```

```
transient). */
        public void recalcular() { valorStock = precio * unidades; }

        public double getValorStock() { return valorStock; }
        public Proveedor getProveedor() { return proveedor; }

        @Override
        public String toString() {
            return String.format("%-9s %-27s %6.2f € x %2d = %7.2f €  (%s)",
                    ref, descripcion, precio, unidades, valorStock,
proveedor.getNombre());
        }
    }

    public static void main(String[] args) {
        File f = new File("datos/inventario_taller.ser");
        f.getParentFile().mkdirs();

        Proveedor shimano = new Proveedor("B-33111222", "Shimano Iberia");
        Proveedor sram = new Proveedor("B-33444555", "SRAM Distribución");
        List<Pieza> inventario = new ArrayList<>();
        inventario.add(new Pieza("CAD-11V", "Cadena 11 velocidades", 32.90, 14,
shimano));
        inventario.add(new Pieza("PAS-HID", "Pastillas freno hidráulico", 18.50, 25,
sram));
        inventario.add(new Pieza("CAS-1142", "Cassette 11-42", 64.00, 6, shimano));
        inventario.add(new Pieza("CUB-29", "Cubierta 29x2.25", 41.75, 10, sram));

        // 1. Toda la lista con UN writeObject (ArrayList es Serializable; sus elementos
también deben serlo)
        try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(f))) {
            oos.writeObject(inventario);
            System.out.println("✅ Inventario guardado: " + inventario.size() + "
piezas");
        } catch (IOException e) {
            System.err.println("Error al guardar: " + e.getMessage());
            return;
        }

        // 2. Lectura: readObject() devuelve Object; se recorre como List<?> y se hace
el cast de cada pieza
        List<Pieza> leidas = new ArrayList<>();
        try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(f))) {
            List<?> lista = (List<?>) ois.readObject();
            for (Object o : lista) {
                leidas.add((Pieza) o);
            }
        } catch (IOException | ClassNotFoundException e) {
            System.err.println("Error al leer: " + e.getMessage());
            return;
        }

        System.out.printf("Recién deserializada, valorStock de %s = %.2f €
(transient)%n",
                leidas.get(0).ref, leidas.get(0).getValorStock());

        double total = 0;
        for (Pieza p : leidas) {
            p.recalcular();                     // reconstruimos el dato derivado
            total += p.getValorStock();
            System.out.println("  " + p);
```

```
        }
        System.out.printf("Valor total del inventario: %.2f €%n", total);

        // 3. La referencia compartida se conserva: las dos piezas de Shimano apuntan al
MISMO objeto
        System.out.println("¿Mismo objeto Proveedor en CAD-11V y CAS-1142? "
                + (leidas.get(0).getProveedor() == leidas.get(2).getProveedor()));
    }
}
```

▶️ Salida real (Java 21, configuración regional española)

```
✅ Inventario guardado: 4 piezas
Recién deserializada, valorStock de CAD-11V = 0,00 € (transient)
  CAD-11V   Cadena 11 velocidades        32,90 € x 14 =  460,60 €  (Shimano Iberia)
  PAS-HID   Pastillas freno hidráulico   18,50 € x 25 =  462,50 €  (SRAM Distribución)
  CAS-1142  Cassette 11-42               64,00 € x  6 =  384,00 €  (Shimano Iberia)
  CUB-29    Cubierta 29x2.25             41,75 € x 10 =  417,50 €  (SRAM Distribución)
Valor total del inventario: 1724,60 €
¿Mismo objeto Proveedor en CAD-11V y CAS-1142? true
```

> **Notas para el aula:** el `0,00 €` de la primera línea es el valor por defecto de un `double` `transient` tras deserializar: el constructor **no** se ejecuta al leer un objeto, así que hay que recalcular a mano. El `true` final muestra que la serialización guarda el **grafo** de objetos: Shimano se escribe una vez y la segunda pieza solo guarda «el objeto número tal» (lo que explica §4.5). Si `Proveedor` no implementara `Serializable`, `writeObject` lanzaría `NotSerializableException: soluciones_b2.InventarioTaller$Proveedor`. Recorrer `List<?>` y hacer el *cast* de cada elemento evita el aviso *unchecked* de `(List<Pieza>) ois.readObject()`, y si el fichero tuviera otra cosa fallaría en esa línea con un `ClassCastException` claro.

Solución Ejercicio 6: Socios con Borrado Lógico y Compactación (`SociosRAF.java`)

```
package soluciones_b2;

import java.io.File;
import java.io.IOException;
import java.io.RandomAccessFile;
import java.nio.file.Files;
import java.nio.file.StandardCopyOption;

public class SociosRAF {
    // id (4) + activo (1) + nombre 15 caracteres (30) + cuota (8) = 43 bytes
    static final int TAM = 43;
    static final int OFF_ACTIVO = 4;
    static final int LONG_NOMBRE = 15;

    private final File fichero;

    public SociosRAF(String ruta) {
```

```
        this.fichero = new File(ruta);
        this.fichero.getParentFile().mkdirs();
    }

    // Texto de longitud fija del apartado 4.3
    static void escribirTextoFijo(RandomAccessFile raf, String texto, int longitud)
throws IOException {
        StringBuilder sb = new StringBuilder(texto);
        sb.setLength(longitud);                              // recorta o rellena con
'\0'
        raf.writeChars(sb.toString().replace('\0', ' '));
    }

    static String leerTextoFijo(RandomAccessFile raf, int longitud) throws IOException {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < longitud; i++) sb.append(raf.readChar());
        return sb.toString().trim();
    }

    /** Alta al final del fichero; devuelve la posición del nuevo registro. */
    public long alta(int id, String nombre, double cuota) throws IOException {
        try (RandomAccessFile raf = new RandomAccessFile(fichero, "rw")) {
            long pos = raf.length() / TAM;
            raf.seek(raf.length());                          // nos colocamos al final
            raf.writeInt(id);
            raf.writeBoolean(true);                          // activo
            escribirTextoFijo(raf, nombre, LONG_NOMBRE);
            raf.writeDouble(cuota);
            return pos;
        }
    }

    /** Baja lógica: solo se sobrescribe el byte de la marca. */
    public void bajaLogica(long pos) throws IOException {
        try (RandomAccessFile raf = new RandomAccessFile(fichero, "rw")) {
            if ((pos + 1) * TAM > raf.length()) {
                System.out.println("❌ No existe la posición " + pos);
                return;
            }
            raf.seek(pos * TAM + OFF_ACTIVO);
            raf.writeBoolean(false);
            System.out.printf("🗑️  Baja lógica de la posición %d (byte %d)%n", pos, pos
* TAM + OFF_ACTIVO);
        }
    }

    public void listar() throws IOException {
        try (RandomAccessFile raf = new RandomAccessFile(fichero, "r")) {
            long total = raf.length() / TAM;
            int activos = 0;
            double cuotas = 0;
            for (long pos = 0; pos < total; pos++) {
                raf.seek(pos * TAM);                         // tras leer un registro ya
estaríamos aquí; seek por claridad
                int id = raf.readInt();
                boolean activo = raf.readBoolean();
                String nombre = leerTextoFijo(raf, LONG_NOMBRE);
                double cuota = raf.readDouble();
                System.out.printf("  [%d] %3d %-15s %6.2f €%s%n", pos, id, nombre,
cuota, activo ? "" : "  (BAJA)");
                if (activo) {
```

```
                    activos++;
                    cuotas += cuota;
                }
            }
            System.out.printf("  %d registros (%d activos) · %d bytes · cuotas activas:
%.2f €%n",
                    total, activos, raf.length(), cuotas);
        }
    }

    /** Copia los registros activos a un temporal y lo pone en lugar del original. */
    public void compactar() throws IOException {
        File temporal = new File(fichero.getPath() + ".tmp");
        int copiados = 0;
        try (RandomAccessFile origen = new RandomAccessFile(fichero, "r");
             RandomAccessFile destino = new RandomAccessFile(temporal, "rw")) {
            destino.setLength(0);                           // por si quedó un temporal
de antes
            long total = origen.length() / TAM;
            for (long pos = 0; pos < total; pos++) {
                origen.seek(pos * TAM);
                int id = origen.readInt();
                boolean activo = origen.readBoolean();
                String nombre = leerTextoFijo(origen, LONG_NOMBRE);
                double cuota = origen.readDouble();
                if (activo) {                               // destino avanza solo:
escritura secuencial
                    destino.writeInt(id);
                    destino.writeBoolean(true);
                    escribirTextoFijo(destino, nombre, LONG_NOMBRE);
                    destino.writeDouble(cuota);
                    copiados++;
                }
            }
        }
        // Ya cerrados los dos: el temporal sustituye al original (apartado 2.2)
        long antes = fichero.length();
        Files.move(temporal.toPath(), fichero.toPath(),
StandardCopyOption.REPLACE_EXISTING);
        System.out.printf("🧹 Compactado: %d registros copiados · %d -> %d bytes%n",
copiados, antes, fichero.length());
    }

    public static void main(String[] args) throws IOException {
        new File("datos/socios.dat").delete();              // empezamos de cero en
cada prueba
        SociosRAF club = new SociosRAF("datos/socios.dat");

        club.alta(101, "Ana Suárez", 35.00);
        club.alta(102, "Guillermo Antuña Vega", 42.50);       // 21 caracteres: se
recorta a 15
        club.alta(103, "Pelayo Rico", 35.00);
        club.alta(104, "Xana Menéndez", 50.00);
        club.alta(105, "Iris Valdés", 42.50);
        System.out.println("Fichero inicial:");
        club.listar();

        club.bajaLogica(1);
        club.bajaLogica(3);
        club.bajaLogica(7);
        System.out.println("Tras las bajas lógicas:");
```

```
        club.listar();

        club.compactar();
        System.out.println("Tras compactar:");
        club.listar();
    }
}
```

▶️ Salida real (Java 21, configuración regional española)

```
Fichero inicial:
  [0] 101 Ana Suárez       35,00 €
  [1] 102 Guillermo Antuñ  42,50 €
  [2] 103 Pelayo Rico      35,00 €
  [3] 104 Xana Menéndez    50,00 €
  [4] 105 Iris Valdés      42,50 €
  5 registros (5 activos) · 215 bytes · cuotas activas: 205,00 €
🗑️  Baja lógica de la posición 1 (byte 47)
🗑️  Baja lógica de la posición 3 (byte 133)
❌ No existe la posición 7
Tras las bajas lógicas:
  [0] 101 Ana Suárez       35,00 €
  [1] 102 Guillermo Antuñ  42,50 €  (BAJA)
  [2] 103 Pelayo Rico      35,00 €
  [3] 104 Xana Menéndez    50,00 €  (BAJA)
  [4] 105 Iris Valdés      42,50 €
  5 registros (3 activos) · 215 bytes · cuotas activas: 112,50 €
🧹 Compactado: 3 registros copiados · 215 -> 129 bytes
Tras compactar:
  [0] 101 Ana Suárez       35,00 €
  [1] 103 Pelayo Rico      35,00 €
  [2] 105 Iris Valdés      42,50 €
  3 registros (3 activos) · 129 bytes · cuotas activas: 112,50 €
```

> **Notas para el aula:** la baja lógica escribe **1 byte** (`writeBoolean` en el offset `pos × 43 + 4`) frente a reescribir todo el fichero; el precio es que el fichero no encoge hasta compactar (215 bytes con 3 activos). La compactación cambia las posiciones (Pelayo pasa de la 2 a la 1): por eso, si otro fichero o un índice guardara posiciones, habría que regenerarlo; buscar por `id` no se ve afectado. El `Files.move` se hace fuera del `try`: con los `RandomAccessFile` aún abiertos, en Windows el `move` falla porque el fichero está bloqueado. «Guillermo Antuña Vega» (21 caracteres) se guarda como «Guillermo Antuñ» (15).